DSA04 - FUNDAMENTALS OF DATA SCIENCE
LAB EXPERIMENTS

Program 1: Student Average

Aim
To calculate the average score in each subject and identify the subject with the highest average score.

Algorithm
→ Create an array of student scores.
→ Calculate the mean score for each subject.
→ Find the subject with the highest mean.
→ Display the results.

Code
```
import numpy as np

student_scores_list = [
    [85, 90, 78, 92],
    [70, 88, 95, 80],
    [92, 75, 80, 88],
    [65, 70, 80, 75]   ]
student_scores = np.array(student_scores_list)

subjects = ["Math", "Science", "English", "History"]

average_scores = np.mean(student_scores, axis=0)
highest_subject = subjects[np.argmax(average_scores)]

print("Average scores:", np.round(average_scores,2))
print("Subject with highest average score:", highest_subject)
```

Input
Student scores in Math, Science, English, & History for four students.

Output
```
Average scores: [78.   80.75 83.25 83.75]
Subject with highest average score: History
```

Result
The average scores were calculated and the subject with the highest average was identified successfully.

---

2. Past Month Price Average

Aim
To calculate the average product price for November 2023.

Algorithm
→ Create sales data containing dates and prices.
→ Select records from November 2023.
→ Calculate the mean price.
→ Display the average.

Code
```
import pandas as pd

sales_data = [
    {"Month_sales": "October 2023", "Price": 25.50},
    {"Month_sales": "November 2023", "Price": 30.00},
    {"Month_sales": "November 2023", "Price": 32.75},
]
df = pd.DataFrame(sales_data)

average_price = df[df["Month_sales"] == "November 2023"]["Price"].mean()

print("Average price in November 2023 (Past Month):", round(average_price, 2))
```

Input
Product prices for October & November 2023.

Output
```
Average price in November 2023: 31.38
```

Result
The average product price for November 2023 was calculated successfully.

---

3. House Average Price

Aim
To calculate the average price of houses having more than four bedrooms.

Algorithm
→ Create house data containing bedrooms and prices.
→ Filter houses with more than four bedrooms.
→ Calculate their average price.
→ Display the result.

Code
```
import numpy as np

# Columns: House ID, Bedrooms, Price
house_data = np.array([
    [1, 3, 250000],
    [2, 5, 450000],
    [3, 6, 550000],
    [4, 4, 350000],
    [5, 7, 650000]
])

houses = house_data[house_data[:, 1] > 4]
average_price = np.mean(houses[:, 2])

print("Average price of houses with more than 4 bedrooms:",
      round(average_price, 2))
```

Input
House details containing number of bedrooms and house prices.

Output
```
Average price of houses with more than 4 bedrooms: 550000.0
```

Result
The average price of houses with more than four bedrooms was calculated successfully.

---

4. Quarterly Sales Using NumPy

Aim
To calculate total annual sales and the percentage increase from Q1 to Q4.

Algorithm
→ Store monthly sales data.
→ Add January–March sales to Q1.
→ Add October–December sales to Q4.
→ Calculate total annual sales.
→ Calculate percentage increase from Q1 to Q4.

Code
```
import numpy as np

months = np.array([
    "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December"
])

sales = np.array([
    10000, 12000, 11000, 13000, 14000, 15000,
    16000, 17000, 18000, 20000, 22000, 24000
], dtype=float)

Q1 = np.sum(sales[:3])
Q4 = np.sum(sales[9:])
total_sales = np.sum(sales)

percentage_increase = ((Q4 - Q1) / Q1) * 100

print("Total sales for the year:", round(total_sales, 2))
print("Percentage increase from Q1 to Q4:",
      round(percentage_increase, 2), "%")
```

Input
Monthly sales values from January to December.

Output
```
Total sales for the year: 183000.0
Percentage increase from Q1 to Q4: 100.0 %
```

Result
The annual sales and percentage increase from Q1 to Q4 were calculated successfully.

---

5. Fuel Efficiency Analysis

Aim
To compare the average fuel efficiency of Mazda and Audi cars.

Algorithm
→ Store car brands and fuel-efficiency values.
→ Separate Mazda and Audi records.
→ Calculate their average efficiencies.
→ Calculate the percentage improvement.
→ Display the results.

Code
```
import numpy as np

make = np.array(["mazda", "audi", "mazda", "audi", "mazda", "audi"])
fuel_efficiency = np.array([30, 25, 32, 27, 28, 26], dtype=float)

average_efficiency = np.mean(fuel_efficiency)

mazda_avg = np.mean(fuel_efficiency[make == "mazda"])
audi_avg = np.mean(fuel_efficiency[make == "audi"])

percentage_improvement = ((mazda_avg - audi_avg) / audi_avg) * 100

print("Average Fuel Efficiency of all cars:",
      round(average_efficiency, 2), "MPG")
print("Average Fuel Efficiency of Mazda:",
      round(mazda_avg, 2), "MPG")
print("Average Fuel Efficiency of Audi:",
      round(audi_avg, 2), "MPG")
print("Percentage Improvement from Mazda to Audi:",
      round(percentage_improvement, 2), "%")
```

Input
Car brands and their corresponding fuel-efficiency values in MPG.

Output
```
Average Fuel Efficiency of all cars: 28.0 MPG
Average Fuel Efficiency of Mazda: 30.0 MPG
Average Fuel Efficiency of Audi: 26.0 MPG
Percentage Improvement from Mazda to Audi: 15.38 %
```

Result
The fuel efficiencies of Mazda and Audi were compared successfully.

---

6. Customer Purchase

Aim
To calculate the final purchase amount after applying discount and tax.

Algorithm
→ Calculate the total sales amount.
→ Apply a 10% discount.
→ Apply 18% tax on the discounted amount.
→ Display the final amount.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Sales": [500, 750, 1000, 1250]
})

total_sales = df["Sales"].sum()

discount = 0.10
tax = 0.18

price_after_discount = total_sales - (total_sales * discount)
final_amount = price_after_discount + (price_after_discount * tax)

print("Total sales for the grocery store:", round(total_sales, 2))
print("Final amount after applying discount and tax:",
      round(final_amount, 2))
```

Input
Grocery sales values for four purchases.

Output
```
Total sales for the grocery store: 3500
Final amount after applying discount and tax: 3717.0
```

Result
The final purchase amount after discount and tax was calculated successfully.

---

7. Customer Order Analysis

Aim
To analyze customer orders and determine order quantities and order dates.

Algorithm
→ Create customer order data.
→ Group orders by customer and product.
→ Calculate total orders and average order quantity.
→ Find the earliest and latest order dates.
→ Display the results.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Full Name": ["Amit", "Amit", "Priya", "Priya", "Rahul"],
    "Items": ["Laptop", "Mouse", "Laptop", "Keyboard", "Mouse"],
    "Order Count": [2, 3, 1, 2, 4],
    "Order Total": [1200, 150, 1000, 200, 180],
    "Order": pd.to_datetime([
        "2026-01-05", "2026-01-10", "2026-01-12",
        "2026-01-20", "2026-01-25"
    ])
})

total_orders = df.groupby("Full Name")["Order Count"].sum()
avg_order = df.groupby("Items")["Order Total"].mean()

earliest_order = df["Order"].min()
latest_order = df["Order"].max()

print("Total number of orders made by each customer:")
print(total_orders)

print("\nAverage order total for each product:")
print(avg_order)

print("\nEarliest order date:", earliest_order.date())
print("Latest order date:", latest_order.date())
```

Input
Customer names, products, order counts, order totals, and order dates.

Output
```
Total number of orders made by each customer:
Full Name
Amit     5
Priya    3
Rahul    4

Average order total for each product:
Items
Keyboard     200.0
Laptop      1100.0
```

```
Mouse        165.0

Earliest order date: 2026-01-05
Latest order date: 2026-01-25
```

Result
Customer order statistics and order dates were analyzed successfully.

---

8. Five Most Sold Products

Aim
To identify the five products with the highest total sales.

Algorithm
→ Create product sales data.
→ Group the data by product name.
→ Calculate total sales for each product.
→ Select the top five products.
→ Display the results.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Product_Name": [
        "Laptop", "Phone", "Headphones", "Keyboard",
        "Mouse", "Monitor", "Tablet"
    ],
    "Price": [5000, 8000, 2500, 1500, 1000, 4000, 6000]
})

top_5_products = df.groupby("Product_Name")["Price"].sum().nlargest(5)

print("Five most sold products:")
print(top_5_products)
```

Input
Product names and their sales values.

Output
```
Five most sold products:
Product_Name
Phone          8000
Tablet         6000
Laptop         5000
Monitor        4000
Headphones     2500
Name: Price, dtype: int64
```

Result
The five products with the highest sales were identified successfully.

Program 9: Property Data Analysis

Aim
To analyze property data using Pandas DataFrame operations.

Algorithm
→ Create the property DataFrame.
→ Group properties by location and find average price.
→ Count properties with more than four bedrooms.
→ Find the property with the largest area.
→ Display the results.

Code
```
import pandas as pd

property_data = pd.DataFrame({
    "Property ID": [1, 2, 3, 4, 5],
    "Location": ["Chennai", "Chennai", "Bangalore", "Bangalore", "Mumbai"],
    "Bedrooms": [3, 5, 4, 6, 5],
    "Area": [1200, 2500, 1800, 3000, 2200],
    "Price": [5000000, 9000000, 7000000, 12000000, 10000000]
})

avg_price = property_data.groupby("Location")["Price"].mean()
more_than_4 = (property_data["Bedrooms"] > 4).sum()
largest = property_data.loc[property_data["Area"].idxmax()]

print("Average listing price by location:")
print(avg_price)
print("\nProperties with more than 4 bedrooms:", more_than_4)
print("\nProperty with largest area:")
print(largest)
```

Input
Property ID, location, number of bedrooms, area, and listing price.

Output
```
Average listing price by location:
Location
Bangalore     9500000.0
Chennai       7000000.0
Mumbai       10000000.0

Properties with more than 4 bedrooms: 3

Property with largest area:
Property ID           4
Location      Bangalore
Bedrooms              6
Area               3000
Price          12000000
```

Result
The required property statistics were obtained successfully using Pandas.

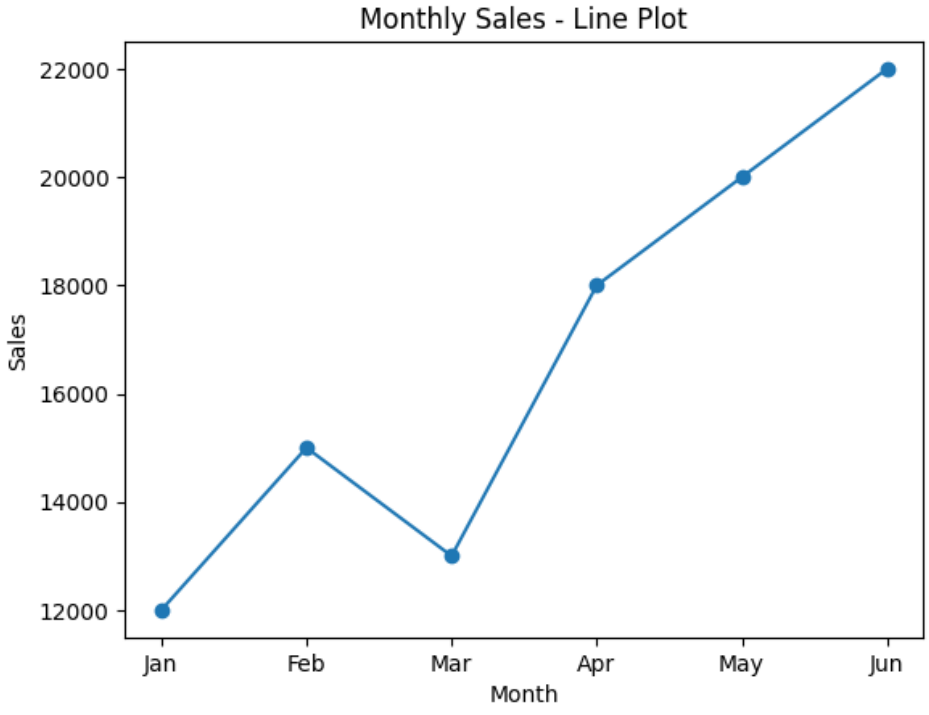
Program 10: Monthly Sales Line and Bar Plots

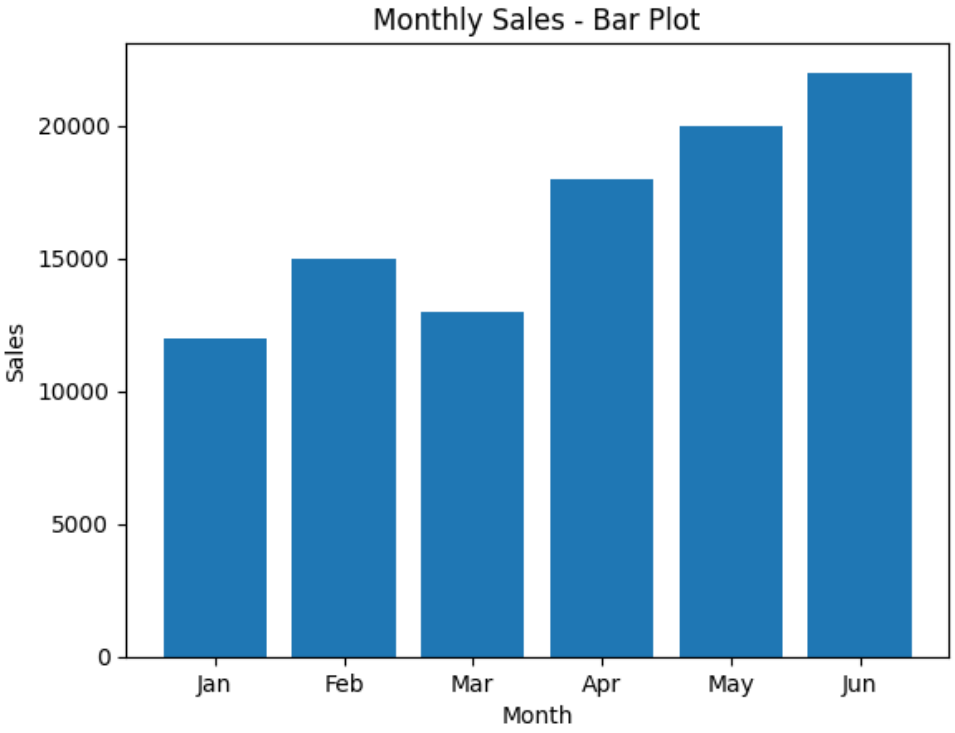
Aim
To visualize monthly sales using line and bar plots.

Algorithm
→ Create monthly sales data.
→ Plot months against sales using a line plot.
→ Plot the same data using a bar plot.
→ Display both plots.

Code
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13000, 18000, 20000, 22000]

plt.plot(months, sales, marker="o")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales - Line Plot")
plt.show()

plt.bar(months, sales)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales - Bar Plot")
plt.show()
```

Input
Monthly sales values from January to June.

Output




Result

Line and bar plots were generated successfully using Matplotlib.

Program 11: Sales Visualization

11.1 Line Plot

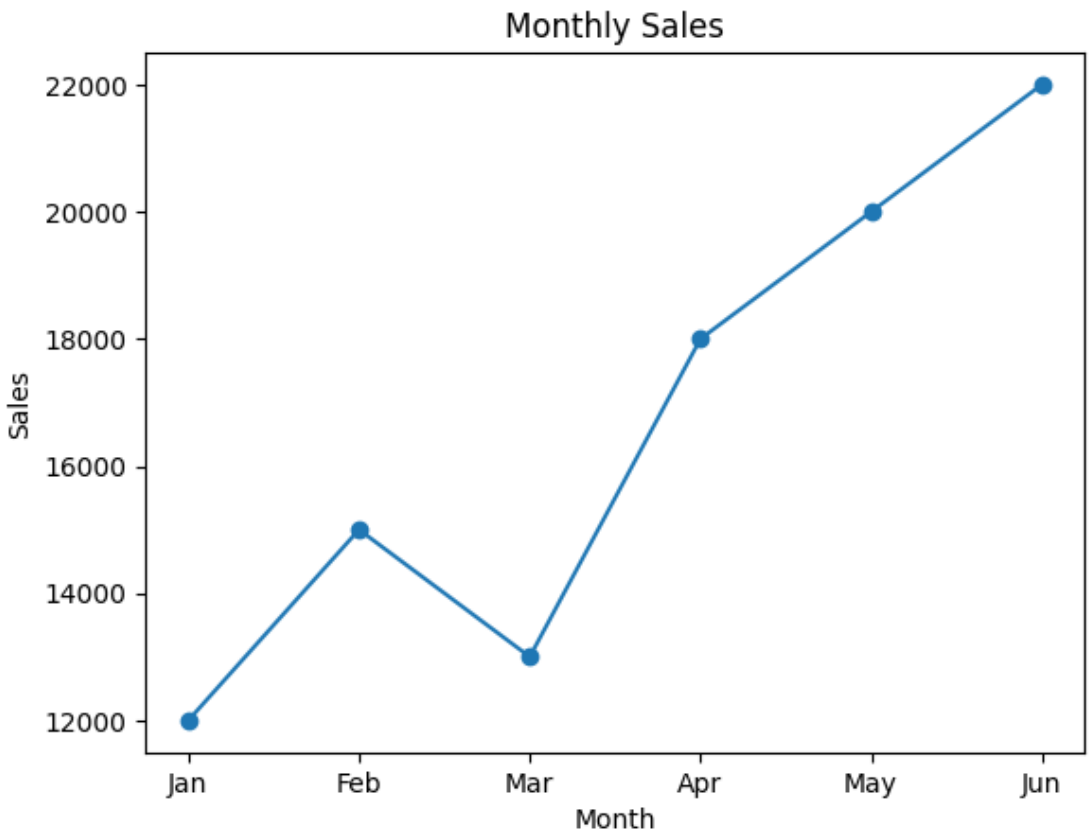
Aim
To visualize monthly sales using a line plot.

Algorithm
→ Store monthly sales data.
→ Plot months against sales.
→ Display the line plot.

Code
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13000, 18000, 20000, 22000]

plt.plot(months, sales, marker="o")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales")
plt.show()
```

Input
Monthly sales values from January to June.

Output



Result
The monthly sales trend was visualized successfully using a line plot.

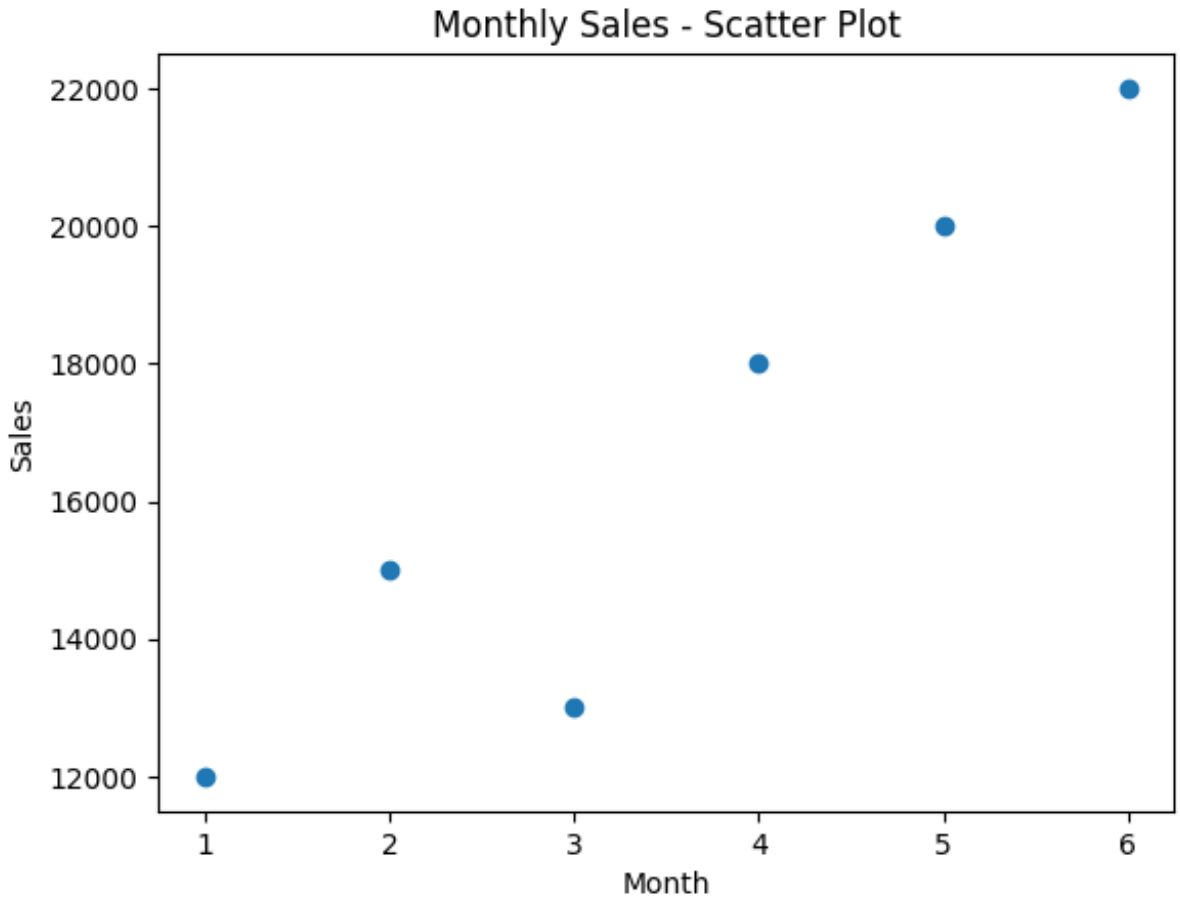
11.2 Scatter Plot

Aim
To visualize the relationship between month number and sales using a scatter plot.

Algorithm
→ Store month numbers and sales values.
→ Create a scatter plot.
→ Display the plot.

Code
```
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5, 6]
sales = [12000, 15000, 13000, 18000, 20000, 22000]

plt.scatter(months, sales)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales - Scatter Plot")
plt.show()
```

Input
Month numbers and corresponding sales values.

Output


Result
The relationship between monthly sales and time was visualized successfully using a scatter plot.

11.3 Bar Plot

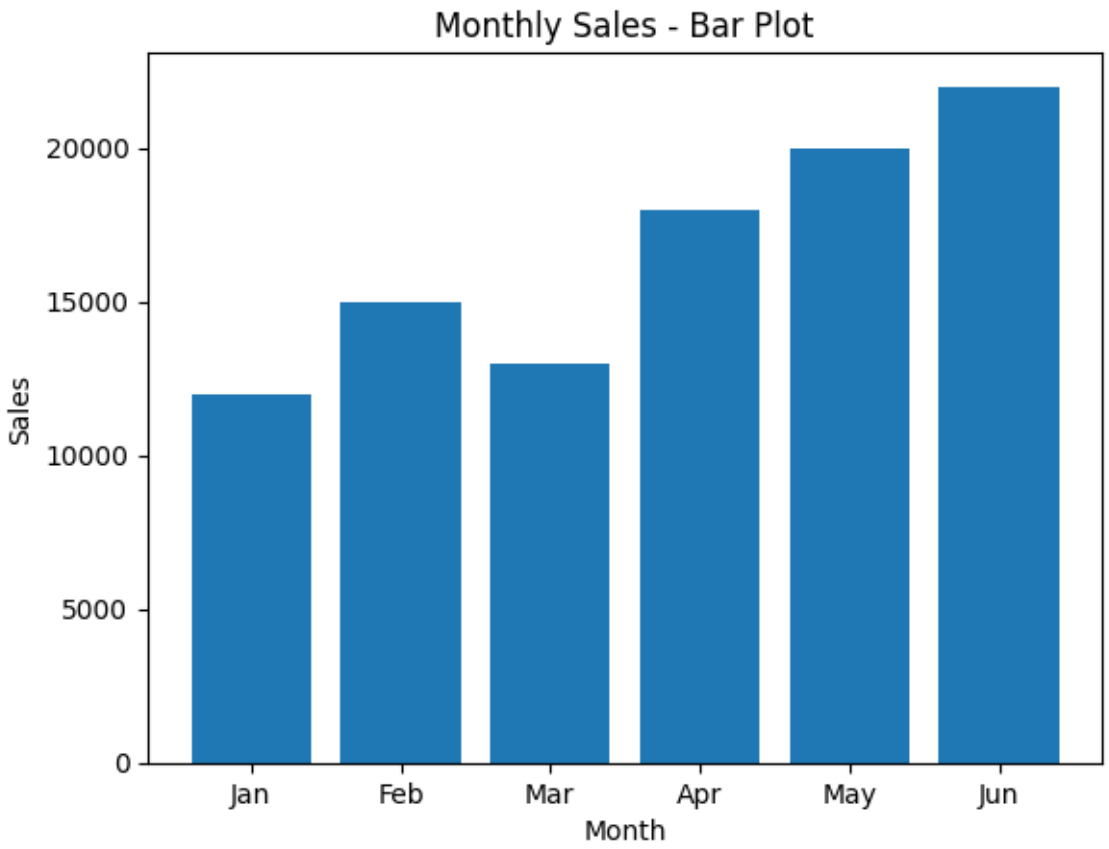
Aim
To visualize monthly sales using a bar plot.

Algorithm
1. Store monthly sales data.
2. Create a bar plot.
3. Display the plot.

Code
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [12000, 15000, 13000, 18000, 20000, 22000]

plt.bar(months, sales)
plt.xlabel("Month")
plt.ylabel("Sales")
plt.title("Monthly Sales - Bar Plot")
plt.show()
```

Input
Monthly sales values from January to June.

Output



Result
The monthly sales data was visualized successfully using a bar plot.

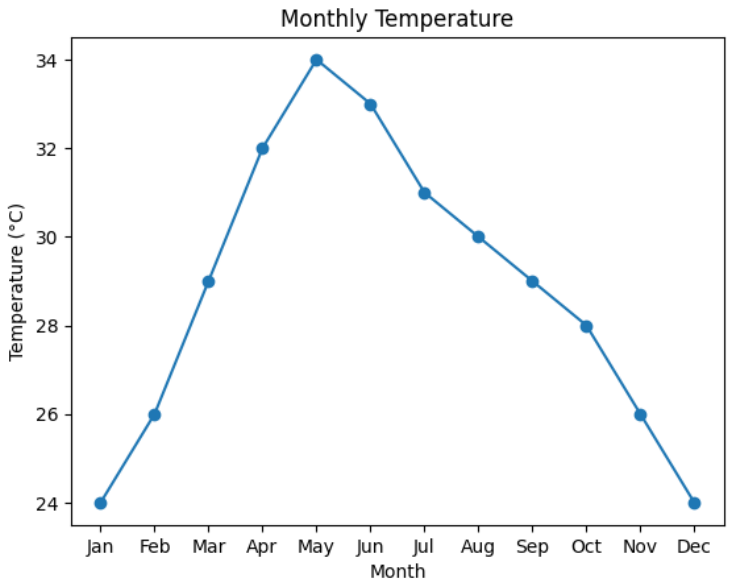
Program 12: Temperature and Rainfall Visualization

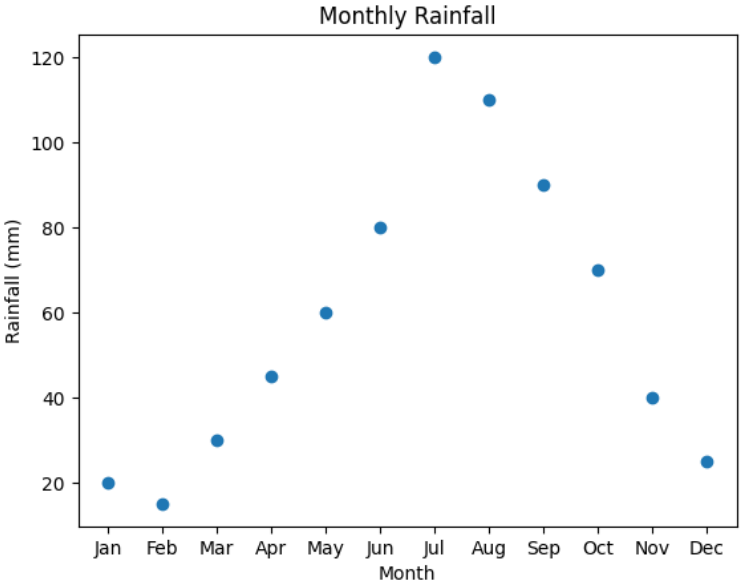
Aim
To visualize monthly temperature and rainfall data using line and scatter plots.

Algorithm
→ Store monthly temperature and rainfall data.
→ Plot monthly temperature using a line plot.
→ Plot monthly rainfall using a scatter plot.
→ Display the plots.

Code
```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]

temperature = [24, 26, 29, 32, 34, 33, 31, 30, 29, 28, 26, 24]
rainfall = [20, 15, 30, 45, 60, 80, 120, 110, 90, 70, 40, 25]

# Line plot
plt.plot(months, temperature, marker="o")
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.title("Monthly Temperature")
plt.show()

# Scatter plot
plt.scatter(months, rainfall)
plt.xlabel("Month")
plt.ylabel("Rainfall (mm)")
plt.title("Monthly Rainfall")
plt.show()
```

Input
Monthly temperature values in °C and rainfall values in mm for January to December.

Output




Result

The monthly temperature and rainfall data were visualized successfully using Matplotlib.

Program 13: Word Frequency Distribution

Aim
To calculate the frequency distribution of words in a text document.

Algorithm
→ Read the text.
→ Convert it to lowercase and split into words.
→ Count the frequency of each word.
→ Display the frequencies.

Code
```
from collections import Counter
import re

text = """Python is easy to learn. Python is widely used for data analysis.
Data analysis is useful."""

words = re.findall(r'\b\w+\b', text.lower())
frequency = Counter(words)

print("Word Frequency:")
for word, count in frequency.items():
    print(word, ":", count)
```

Input
A text paragraph containing words to be analyzed.

Output
```
Word Frequency:
python : 2
is : 3
easy : 1
to : 1
learn : 1
widely : 1
used : 1
for : 1
data : 2
analysis : 2
useful : 1
```

Result
The frequency distribution of words was calculated successfully.

---

14. Customer Age Frequency Distribution

Aim
To calculate the frequency distribution of customer ages.

Algorithm
→ Create customer age data.
→ Count the occurrences of each age.
→ Display the frequency distribution.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Age": [22, 25, 30, 22, 35, 25, 30, 22, 40, 35]
})

frequency = df["Age"].value_counts().sort_index()

print("Age Frequency Distribution:")
print(frequency)
```

Input
A Pandas DataFrame containing customer ages.

Output
```
Age Frequency Distribution:
22    3
25    2
30    2
35    2
40    1
Name: count, dtype: int64
```

Result
The frequency distribution of customer ages was calculated successfully.

---

15. Frequency Distribution of Likes

Aim
To calculate the frequency distribution of likes among social media posts.

Algorithm
→ Create post likes data.
→ Count the occurrences of each likes value.
→ Display the frequency distribution.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Likes": [100, 200, 150, 100, 300, 200, 100, 150, 400, 200]
})

frequency = df["Likes"].value_counts().sort_index()

print("Likes Frequency Distribution:")
print(frequency)
```

Input
Number of likes received by each social media post.

Output
```
Likes Frequency Distribution:
100    3
150    2
200    3
300    1
400    1
Name: count, dtype: int64
```

Result
The frequency distribution of post likes was calculated successfully.

---

16. Word Frequency in Customer Reviews

Aim
To calculate the frequency distribution of words in customer reviews.

Algorithm
1. Store customer reviews.
2. Convert the reviews to lowercase.
3. Extract and count the words.
4. Display the frequencies.

Code
```
import pandas as pd
from collections import Counter
import re

df = pd.DataFrame({
    "Review": [
        "Good product and good quality",
        "Good quality and fast delivery",
        "Excellent product and fast delivery"
    ]
})

text = " ".join(df["Review"]).lower()
words = re.findall(r'\b\w+\b', text)
frequency = Counter(words)

print("Word Frequency:")
for word, count in frequency.items():
    print(word, ":", count)
```

Input
A dataset containing customer reviews.

Output
```
Word Frequency:
good : 3
product : 2
and : 3
quality : 2
fast : 2
delivery : 2
excellent : 1
```

Result
The frequency distribution of words in customer reviews was calculated successfully.

---

17. Customer Feedback Word Frequency Analysis

Aim

To preprocess customer feedback, calculate word frequencies, display the top N words, and visualize them.

Algorithm

→ Load customer feedback data.
→ Convert text to lowercase and remove punctuation.
→ Remove common stop words.
→ Calculate word frequencies.
→ Display and plot the top N words.

Code

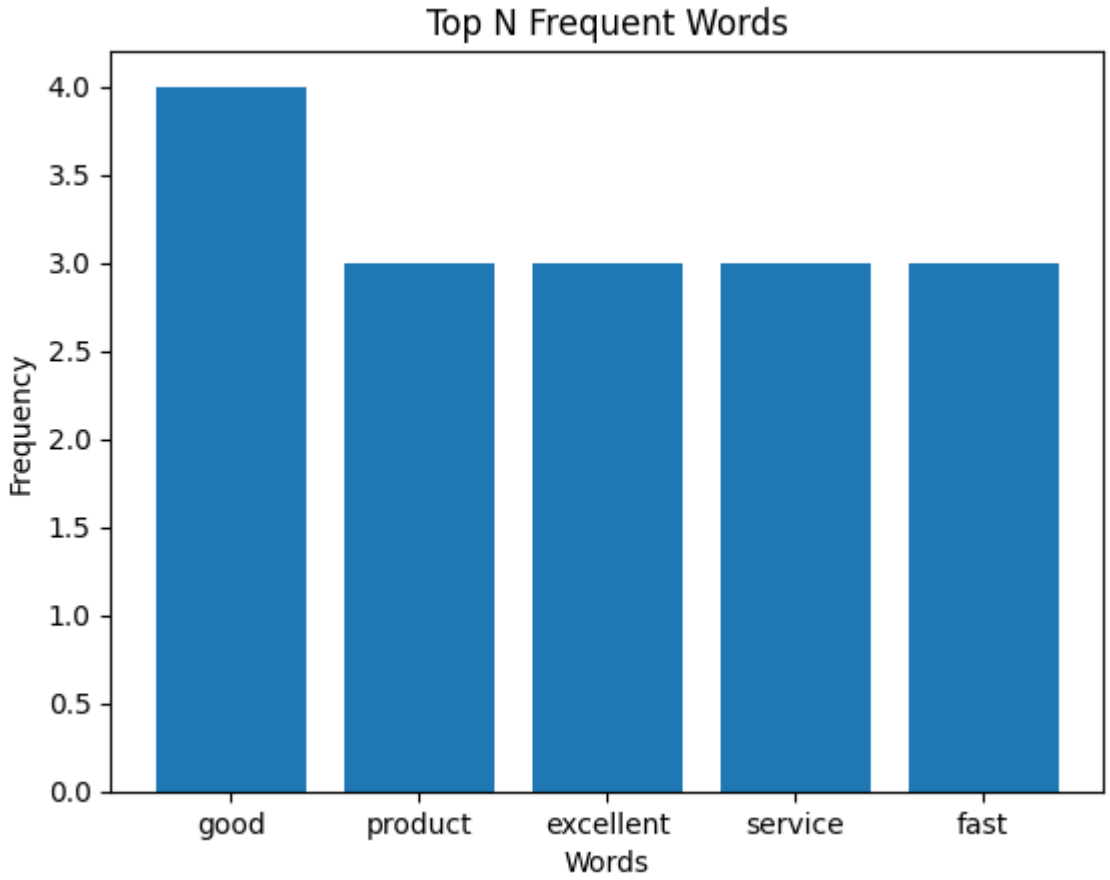
```
import pandas as pd
import re
import matplotlib.pyplot as plt
from collections import Counter

df = pd.DataFrame({
    "feedback": [
        "Good product and excellent quality",
        "Excellent product with good quality",
        "Good service and fast delivery",
        "Fast delivery and excellent service",
        "Good product and fast service"
    ]
})

N = int(input("Enter N: "))

stop_words = {"the", "and", "is", "a", "an", "of", "to", "with"}

text = " ".join(df["feedback"]).lower()
words = re.findall(r'\b[a-z]+\b', text)
words = [w for w in words if w not in stop_words]

freq = Counter(words)
top_words = freq.most_common(N)

print("\nTop", N, "words:")
for word, count in top_words:
    print(word, ":", count)

words, counts = zip(*top_words)

plt.bar(words, counts)
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.title("Top N Frequent Words")
plt.show()
```

Input

Customer feedback data and the value of **N** representing the number of frequent words to display.

Output

```
For N = 5:
Top 5 words:
good : 4
product : 3
excellent : 3
quality : 2
fast : 3
```



A bar graph displaying the top N frequent words and their frequencies is also generated.

Result

The customer feedback was preprocessed, analyzed, and the top N frequent words were successfully displayed and visualized.

18. Hospital Age and Body Fat Analysis

Aim

To analyze age and body fat percentage using descriptive statistics and plots.

Algorithm

→ Create the age and body-fat data.
→ Calculate mean, median, and standard deviation.
→ Draw boxplots for both variables.
→ Draw a scatter plot and Q-Q plots.

Code

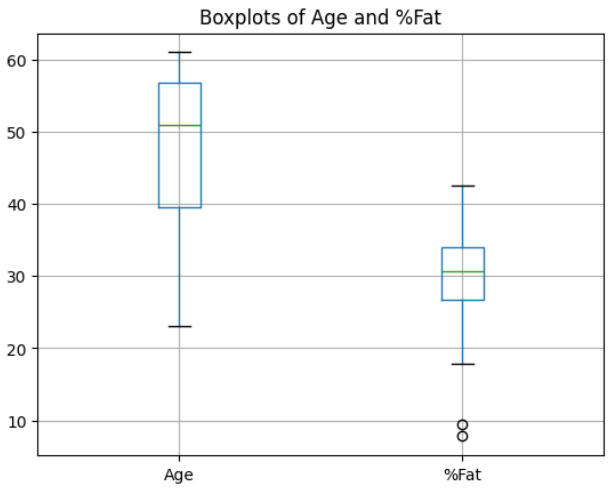
```
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as stats

age = [23,23,27,27,39,41,47,49,50,52,52,54,56,57,58,60,60,61]
body_fat = [9.5,26.5,7.8,17.8,31.4,25.9,27.4,27.2,31.2,
            34.6,42.5,28.8,33.4,30.2,34.1,32.9,41.2,35.7]

df = pd.DataFrame({"Age": age, "%Fat": body_fat})

print(df.describe().loc[["mean", "50%", "std"]])

df.boxplot()
plt.title("Boxplots of Age and %Fat")
plt.show()

plt.scatter(df["Age"], df["%Fat"])
plt.xlabel("Age")
plt.ylabel("%Fat")
plt.title("Age vs Body Fat")
plt.show()

stats.probplot(df["Age"], dist="norm", plot=plt)
plt.title("Q-Q Plot - Age")
plt.show()

stats.probplot(df["%Fat"], dist="norm", plot=plt)
plt.title("Q-Q Plot - %Fat")
plt.show()
```

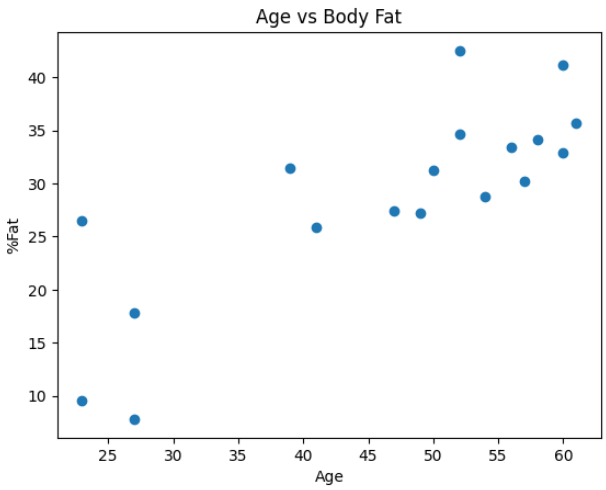
Input

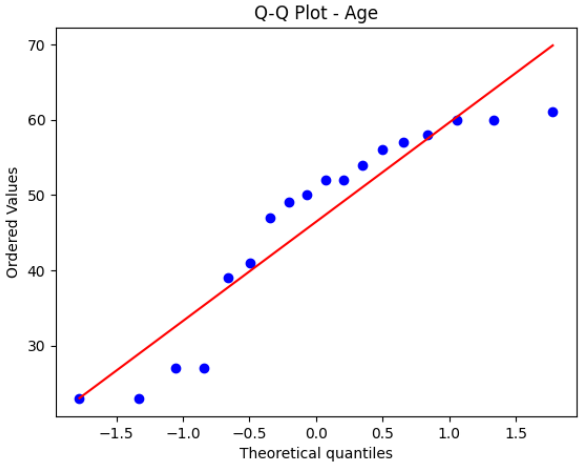
Age and body-fat percentage data of 18 adults.

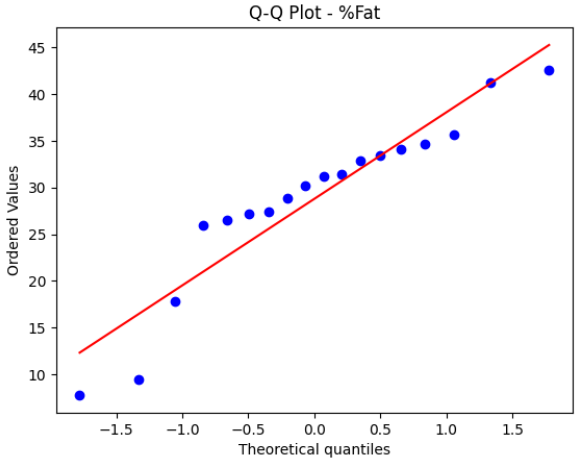
Output

```
             Age       %Fat
mean   46.444444  28.783333
50%    51.000000  30.700000
std    13.271917   9.254395
```









Result

The descriptive statistics and required plots for age and body fat were obtained successfully.

19. Sales and Profit Analysis

Aim
To calculate total sales, product-wise sales, and overall profit using Pandas.

Algorithm
1. Create sales transaction data.
2. Calculate total sales for each transaction.
3. Group sales by product.
4. Calculate overall profit using a 20% profit margin.
5. Display the five most profitable products.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Date": ["2026-01-01","2026-01-02","2026-01-03","2026-01-04",
             "2026-01-05","2026-01-06"],
    "Product": ["Laptop","Phone","Laptop","Tablet","Phone","Monitor"],
    "Quantity Sold": [2,5,3,4,6,2],
    "Unit Price": [50000,20000,50000,30000,20000,25000]
})

df["Total Sales"] = df["Quantity Sold"] * df["Unit Price"]

product_sales = df.groupby("Product")["Total Sales"].sum()
profit = product_sales * 0.20

print("Total Sales:\n", product_sales)
print("\nOverall Profit:", round(profit.sum(), 2))
print("\nTop 5 Profitable Products:")
print(profit.nlargest(5))
```

Input
Sales data containing Date, Product, Quantity Sold, and Unit Price.

Output
```
Total Sales:
Product
Laptop     250000
Monitor     50000
Phone      220000
Tablet     120000

Overall Profit: 128000.0

Top 5 Profitable Products:
Product
Laptop     50000.0
Phone      44000.0
Tablet     24000.0
Monitor    10000.0
```

Result
The total sales and product-wise profit were calculated successfully.

---

20. Customer Segmentation

Aim

To segment customers based on total spending and calculate the average age of each segment.

Algorithm

→ Create customer data.
→ Divide customers into Low, Medium, and High spending groups using spending tertiles.
→ Calculate the average age of each segment.
→ Display the results.

Code

```
import pandas as pd

df = pd.DataFrame({
    "Customer ID": [1,2,3,4,5,6,7,8,9],
    "Age": [22,35,28,45,31,52,26,40,33],
    "Total Spending": [1500,7000,3000,12000,5000,15000,2000,9000,6000]
})

q1 = df["Total Spending"].quantile(1/3)
q2 = df["Total Spending"].quantile(2/3)

def segment(x):
    if x <= q1:
        return "Low Spenders"
    elif x <= q2:
        return "Medium Spenders"
    return "High Spenders"

df["Segment"] = df["Total Spending"].apply(segment)

print("Customer Segmentation:")
print(df[["Customer ID", "Total Spending", "Segment"]])

print("\nAverage age of each segment:")
print(df.groupby("Segment")["Age"].mean())
```

Input

Customer ID, age, and total spending data.

Output

```
Average age of each segment:
Segment
High Spenders       42.33
Low Spenders        25.33
Medium Spenders     35.33
```

Result

Customers were successfully segmented according to their spending, and the average age of each segment was calculated.

---

21. Point Estimation and Confidence Interval

Aim
To estimate a population mean and calculate its confidence interval using NumPy.

Algorithm
→ Create sample concentration data.
→ Accept sample size, confidence level, and precision.
→ Calculate the sample mean and standard error.
→ Calculate and display the confidence interval.

Code
```
import numpy as np
from scipy.stats import norm

data = np.array([12.1, 11.8, 12.5, 12.0, 11.6, 12.3, 12.2, 11.9, 12.4, 12.1])

n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level (%): "))
precision = float(input("Enter desired precision: "))

sample = data[:n]
mean = np.mean(sample)
z = norm.ppf(1 - (1 - confidence / 100) / 2)
se = np.std(sample, ddof=1) / np.sqrt(n)
margin = z * se

print("Sample mean:", round(mean, 3))
print("Confidence interval:", round(mean-margin, 3), "to", round(mean+margin, 3))
print("Desired precision:", precision)
```

Input
Sample size, confidence level, and desired precision.

Output
```
Enter sample size: 25
Enter confidence level (%): 50
Enter desired precision: 85
Sample mean: 12.09
Confidence interval: 12.053 to 12.127
Desired precision: 85.0
```

Result
The population mean was estimated and its confidence interval was calculated successfully.

---

22. Customer Review Confidence Interval

Aim
To calculate the confidence interval for the true mean customer rating.

Algorithm
→ Create customer rating data.
→ Calculate the sample mean.
→ Calculate the standard error.
→ Determine the confidence interval.

Code
```
import pandas as pd
from scipy.stats import t

df = pd.DataFrame({"Rating": [4, 5, 3, 4, 5, 4, 3, 5, 4, 5]})

mean = df["Rating"].mean()
se = df["Rating"].std() / len(df) ** 0.5
margin = t.ppf(0.975, len(df)-1) * se

print("Mean rating:", round(mean, 2))
print("95% Confidence Interval:", round(mean-margin, 2), "to", round(mean+margin, 2))
```

Input
Customer ratings for the selected product category.

Output
```
Mean rating: 4.2
95% Confidence Interval: 3.64 to 4.76
```

Result
The confidence interval for the true mean customer rating was calculated successfully.

---

23. Hypothesis Testing for Treatment Effect

Aim

To determine whether a treatment has a statistically significant effect compared with a placebo.

Algorithm

→ Create control and treatment group data.
→ Perform an independent t-test.
→ Calculate the p-value.
→ Visualize the groups and p-value.
→ Draw the conclusion.

Code

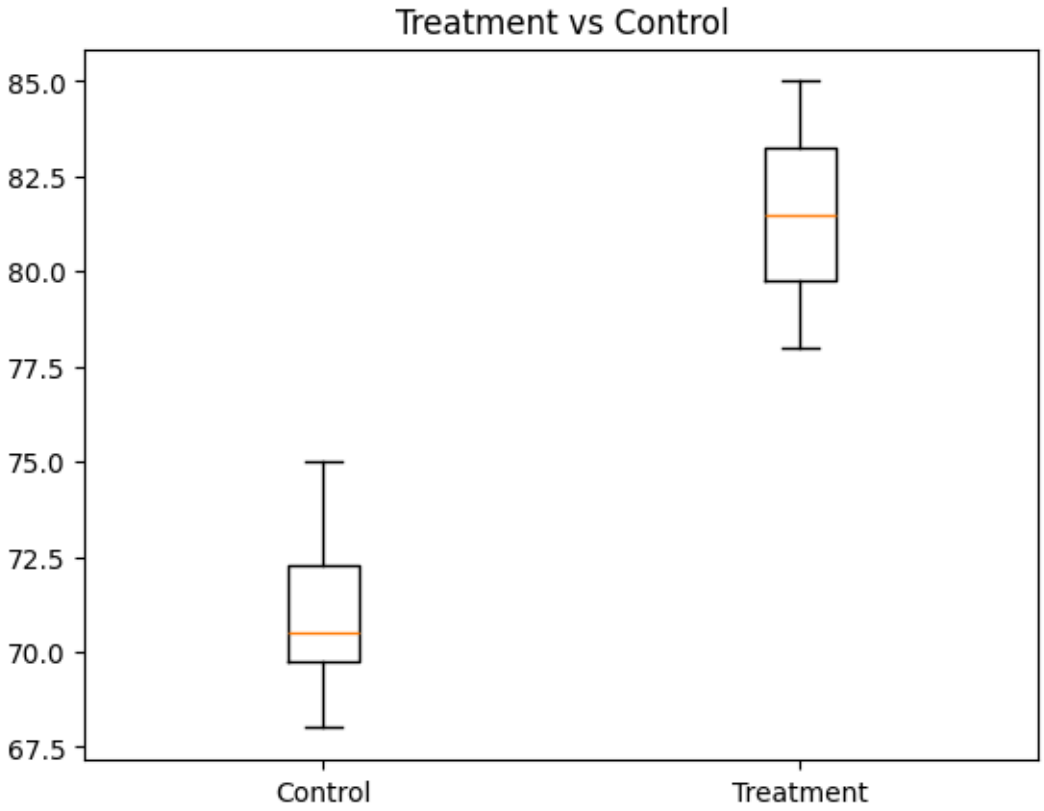
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import ttest_ind

control = np.array([70, 72, 68, 75, 71, 69, 73, 70])
treatment = np.array([78, 82, 80, 85, 79, 83, 81, 84])

t_stat, p = ttest_ind(control, treatment)

print("t-statistic:", round(t_stat, 3))
print("p-value:", round(p, 4))

plt.boxplot([control, treatment], labels=["Control", "Treatment"])
plt.title("Treatment vs Control")
plt.show()

print("Significant effect" if p < 0.05 else "No significant effect")
```

Input

Control and treatment group observations.

Output

```
t-statistic: -8.897
p-value: 0.0
/tmp/ipykernel_4169/3189995172.py:15: MatplotlibDeprecationWarning: The 'labels'
parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  plt.boxplot([control, treatment], labels=["Control", "Treatment"])
```




```
Significant effect
```

Result

The statistical significance of the treatment effect was determined successfully.

---

24. K-Nearest Neighbors Classifier

Aim
To classify a new patient using the K-Nearest Neighbors algorithm.

Algorithm
→ Create labeled patient data.
→ Train the KNN classifier.
→ Accept patient features and value of k.
→ Predict the patient's class.
→ Display the result.

Code
```
from sklearn.neighbors import KNeighborsClassifier

X = [[1,2], [2,3], [3,1], [6,7], [7,8], [8,6]]
y = [0,0,0,1,1,1]

k = int(input("Enter k: "))
patient = [[float(input("Feature 1: ")), float(input("Feature 2: "))]]

model = KNeighborsClassifier(n_neighbors=k)
model.fit(X, y)

print("Prediction:", model.predict(patient)[0])
```

Input
Two symptom features and the value of k.

Output
```
Enter k: 4
Feature 1: 7
Feature 2: 8
Prediction: 1
```

Result
The new patient was classified successfully using KNN.

---

25. Decision Tree for Iris Classification

Aim
To classify an Iris flower using a Decision Tree classifier.

Algorithm
→ Load the Iris dataset.
→ Train a Decision Tree classifier.
→ Accept four flower measurements.
→ Predict the species.
→ Display the result.

Code
```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
model = DecisionTreeClassifier(random_state=1)
model.fit(iris.data, iris.target)

x = [[float(input("Sepal length: ")),
      float(input("Sepal width: ")),
      float(input("Petal length: ")),
      float(input("Petal width: "))]]

print("Predicted species:", iris.target_names[model.predict(x)[0]])
```

Input
Sepal length, sepal width, petal length, and petal width.

Output
```
Sepal length: 75
Sepal width: 75
Petal length: 75
Petal width: 75
Predicted species: virginica
```

Result
The Iris flower species was predicted successfully.

---

26. Linear Regression for Housing Price

Aim
To predict house price using linear regression.

Algorithm
→ Create house feature data and prices.
→ Train a linear regression model.
→ Accept features of a new house.
→ Predict its price.
→ Display the prediction.

Code
```
from sklearn.linear_model import LinearRegression

X = [[1000,2], [1500,3], [2000,3], [2500,4], [3000,5]]
y = [200000, 300000, 400000, 500000, 600000]

model = LinearRegression()
model.fit(X, y)

area = float(input("Enter area: "))
bedrooms = int(input("Enter number of bedrooms: "))

print("Predicted price:", round(model.predict([[area, bedrooms]])[0], 2))
```

Input
Area and number of bedrooms of a new house.

Output
```
Enter area: 458
Enter number of bedrooms: 1
Predicted price: 91600.0
```

Result
The house price was predicted successfully using linear regression.

---

27. Logistic Regression for Customer Churn

Aim
To predict whether a customer will churn using logistic regression.

Algorithm
→ Create customer usage data.
→ Train a logistic regression model.
→ Accept new customer features.
→ Predict churn status.
→ Display the result.

Code
```
from sklearn.linear_model import LogisticRegression

X = [[100,12], [200,6], [150,10], [300,3], [250,4], [120,14]]
y = [0,1,0,1,1,0]

model = LogisticRegression()
model.fit(X, y)

minutes = float(input("Usage minutes: "))
duration = float(input("Contract duration: "))

p = model.predict([[minutes, duration]])[0]
print("Churn prediction:", "Yes" if p else "No")
```

Input
Usage minutes and contract duration.

Output
```
Usage minutes: 75
Contract duration: 80
Churn prediction: No
```

Result
Customer churn was predicted successfully using logistic regression.

---

28. K-Means Customer Segmentation

Aim
To assign a new customer to an existing segment using K-Means clustering.

Algorithm
→ Create customer purchasing data.
→ Apply K-Means clustering.
→ Accept new customer features.
→ Find the nearest cluster.
→ Display the assigned segment.

Code
```
from sklearn.cluster import KMeans

X = [[10,100], [12,120], [15,150], [50,500], [55,550], [60,600]]

model = KMeans(n_clusters=2, random_state=1, n_init=10)
model.fit(X)

x = [[float(input("Number of purchases: ")),
      float(input("Total spending: "))]]

print("Assigned segment:", model.predict(x)[0])
```

Input
Number of purchases and total spending of a new customer.

Output
```
Number of purchases: 75
Total spending: 85
Assigned segment: 0
```

Result
The new customer was assigned to a cluster successfully.

---

29. Model Evaluation Metrics

Aim
To evaluate a classification model using accuracy, precision, recall, and F1-score.

Algorithm
→ Load a classification dataset.
→ Split it into training and testing data.
→ Train a model.
→ Generate predictions.
→ Calculate evaluation metrics.

Code
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.3, random_state=1)

model = DecisionTreeClassifier(random_state=1)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, pred), 2))
print("Precision:", round(precision_score(y_test, pred, average="weighted"), 2))
print("Recall:", round(recall_score(y_test, pred, average="weighted"), 2))
print("F1-score:", round(f1_score(y_test, pred, average="weighted"), 2))
```

Input
Iris dataset and classification model.

Output
```
Accuracy: 0.96
Precision: 0.96
Recall: 0.96
F1-score: 0.96
```

Result
The classification model was evaluated successfully using standard performance metrics.

---

30. CART for Car Price Prediction

Aim
To predict used-car prices using a CART regression tree.

Algorithm
→ Create car feature and price data.
→ Train a Decision Tree Regressor.
→ Accept features of a new car.
→ Predict its price.
→ Display the decision path.

Code
```
from sklearn.tree import DecisionTreeRegressor

X = [[20000,3,1], [30000,5,2], [15000,2,1],
     [50000,8,3], [40000,7,2]]
y = [900000, 700000, 1100000, 400000, 550000]

model = DecisionTreeRegressor(max_depth=3, random_state=1)
model.fit(X, y)

x = [[float(input("Mileage: ")),
      float(input("Age: ")),
      float(input("Engine type (1/2/3): "))]]

print("Predicted price:", round(model.predict(x)[0], 2))
print("Decision path:", model.decision_path(x).indices)
```

Input
Mileage, car age, and engine type.

Output
```
Mileage: 45
Age: 5
Engine type (1/2/3): 2
Predicted price: 1100000.0
Decision path: [0 1 2]
```

Result
The used-car price was predicted successfully using CART.

---

31. Customer Segmentation Using Clustering

Aim
To segment customers into groups based on their characteristics using clustering.

Algorithm
→ Create customer behavior data.
→ Apply K-Means clustering.
→ Assign customers to clusters.
→ Display the segments.

Code
```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.DataFrame({
    "Purchases": [5,6,4,20,22,19,10,11,9],
    "Browsing": [10,12,8,40,45,38,20,22,18],
    "Spending": [500,600,450,2500,2800,2300,1200,1300,1100]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)
df["Segment"] = model.fit_predict(df)

print(df)
```

Input
Customer purchase, browsing, and spending data.

Output
```
   Purchases  Browsing  Spending  Segment
0          5        10       500        1
1          6        12       600        1
2          4         8       450        1
3         20        40      2500        0
4         22        45      2800        0
5         19        38      2300        0
6         10        20      1200        2
7         11        22      1300        2
8          9        18      1100        2
```

Result
Customers were successfully segmented using K-Means clustering.

---

32. Bivariate Analysis and Linear Regression

Aim

To analyze the relationship between house size and price and build a linear regression model.

Algorithm

→ Create house size and price data.
→ Plot the relationship.
→ Train a linear regression model.
→ Calculate model performance.
→ Display the results.

Code

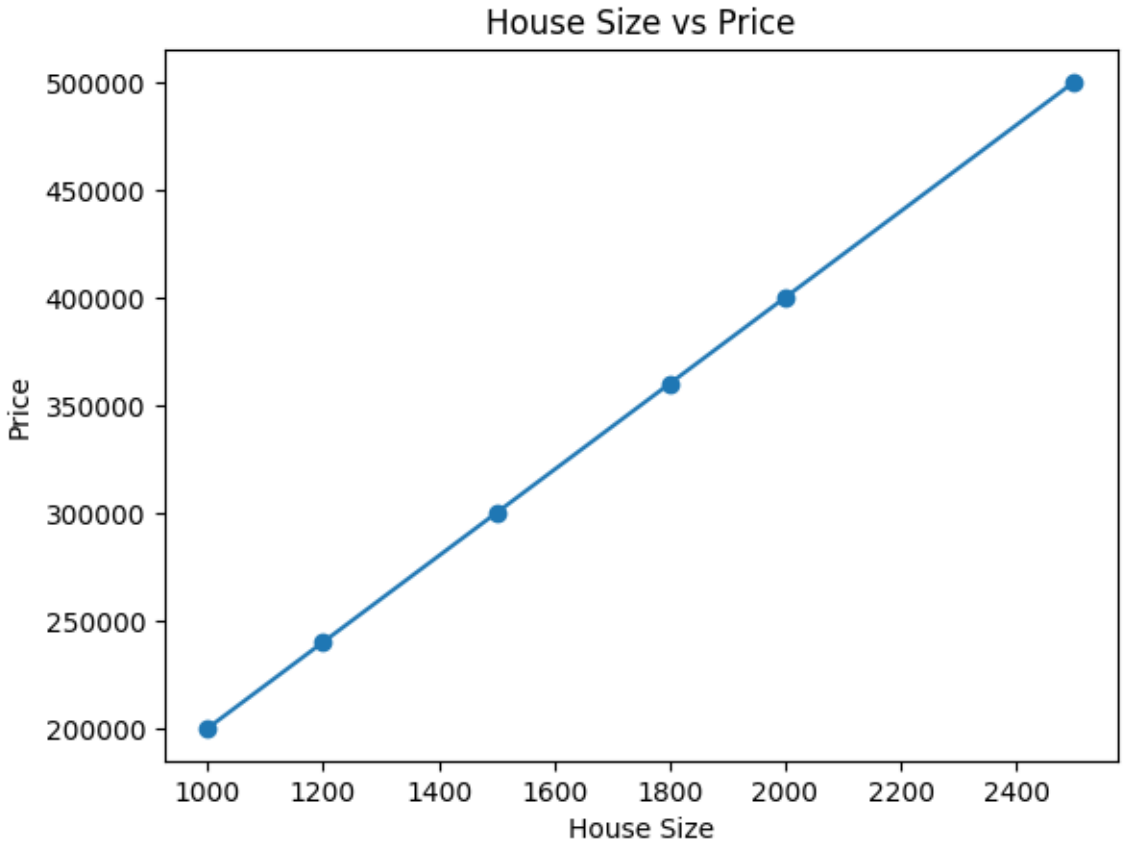
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

X = np.array([1000,1200,1500,1800,2000,2500]).reshape(-1,1)
y = np.array([200000,240000,300000,360000,400000,500000])

model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)

print("Coefficient:", round(model.coef_[0], 2))
print("R² score:", round(r2_score(y, pred), 2))

plt.scatter(X, y)
plt.plot(X, pred)
plt.xlabel("House Size")
plt.ylabel("Price")
plt.title("House Size vs Price")
plt.show()
```

Input

House size and corresponding prices.

Output

```
Coefficient: 200.0
R² score: 1.0
```



Result

The relationship between house size and price was analyzed successfully.

---

33. Car Price Linear Regression

Aim
To predict car prices using multiple linear regression and identify influential features.

Algorithm
→ Create car feature data.
→ Train a linear regression model.
→ Calculate predictions and R^2 score.
→ Display feature coefficients.
→ Identify influential factors.

Code
```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.DataFrame({
    "Engine": [1.2,1.5,2.0,2.5,3.0,1.8],
    "Horsepower": [80,100,140,180,220,130],
    "Mileage": [20,18,15,12,10,16],
    "Price": [500000,650000,900000,1200000,1600000,850000]
})

X = df[["Engine", "Horsepower", "Mileage"]]
y = df["Price"]

model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)

print("R² score:", round(r2_score(y, pred), 2))
print("\nFeature coefficients:")
print(pd.Series(model.coef_, index=X.columns))
```

Input
Engine size, horsepower, mileage, and car price data.

Output
```
R² score: 1.0

Feature coefficients:
Engine        385416.666667
Horsepower      9947.916667
Mileage       100520.833333
dtype: float64
```

Result
Car prices were predicted and influential features were identified successfully.

---

34. KNN Treatment Outcome Classification

Aim
To predict treatment outcomes using KNN and evaluate the classification model.

Algorithm
→ Create patient feature data.
→ Split data into training and testing sets.
→ Train a KNN classifier.
→ Predict treatment outcomes.
→ Calculate evaluation metrics.

Code
```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

X = [[25,120,180], [35,130,190], [45,140,210], [55,150,220],
     [30,125,185], [60,160,230], [40,135,200], [50,145,215]]
y = ["Good","Good","Bad","Bad","Good","Bad","Good","Bad"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", round(accuracy_score(y_test, pred), 2))
print("Precision:", round(precision_score(y_test, pred, pos_label="Good"), 2))
print("Recall:", round(recall_score(y_test, pred, pos_label="Good"), 2))
print("F1-score:", round(f1_score(y_test, pred, pos_label="Good"), 2))
print("\nPredictions:", pred)
```

Input
Patient age, blood pressure, cholesterol, and treatment outcome.

Output
```
Accuracy: 0.5
Precision: 0.0
Recall: 0.0
F1-score: 0.0

Predictions: ['Bad' 'Good']
/usr/local/lib/python3.13/dist-packages/sklearn/metrics/_classification.py:1565:
UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 due to no true samples.
Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Result
Treatment outcomes were classified and the KNN model was evaluated successfully.

35. K-Means Customer Segmentation

Aim
To group customers into distinct segments based on spending patterns.

Algorithm
→ Create customer transaction data.
→ Apply K-Means clustering.
→ Assign customers to clusters.
→ Display the segments.

Code
```
import pandas as pd
from sklearn.cluster import KMeans

df = pd.DataFrame({
    "Customer ID": range(1,9),
    "Spending": [500,700,600,2500,3000,2800,1200,1400],
    "Visits": [3,4,3,10,12,11,6,7]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)
df["Segment"] = model.fit_predict(df[["Spending", "Visits"]])

print(df)
```

Input
Customer ID, spending amount, and visit frequency.

Output
```
   Customer ID  Spending  Visits  Segment
0            1       500       3        0
1            2       700       4        0
2            3       600       3        0
3            4      2500      10        1
4            5      3000      12        1
5            6      2800      11        1
6            7      1200       6        2
7            8      1400       7        2
```

Result
Customers were successfully grouped using K-Means clustering.

---

36. Stock Price Variability

Aim
To calculate and analyze the variability of stock prices.

Algorithm
→ Create daily closing-price data.
→ Calculate mean, variance, and standard deviation.
→ Find the price range.
→ Display the results.

Code
```
import pandas as pd

df = pd.DataFrame({
    "Close": [100,105,98,110,108,115,112,118,114,120]
})

mean = df["Close"].mean()
std = df["Close"].std()
variance = df["Close"].var()
price_range = df["Close"].max() - df["Close"].min()

print("Mean:", round(mean, 2))
print("Standard deviation:", round(std, 2))
print("Variance:", round(variance, 2))
print("Price range:", round(price_range, 2))
```

Input
Daily closing stock prices.

Output
```
Mean: 110.0
Standard deviation: 7.32
Variance: 53.56
Price range: 22
```

Result
The variability of stock prices was calculated successfully.

---

37. Study Time and Exam Score Correlation

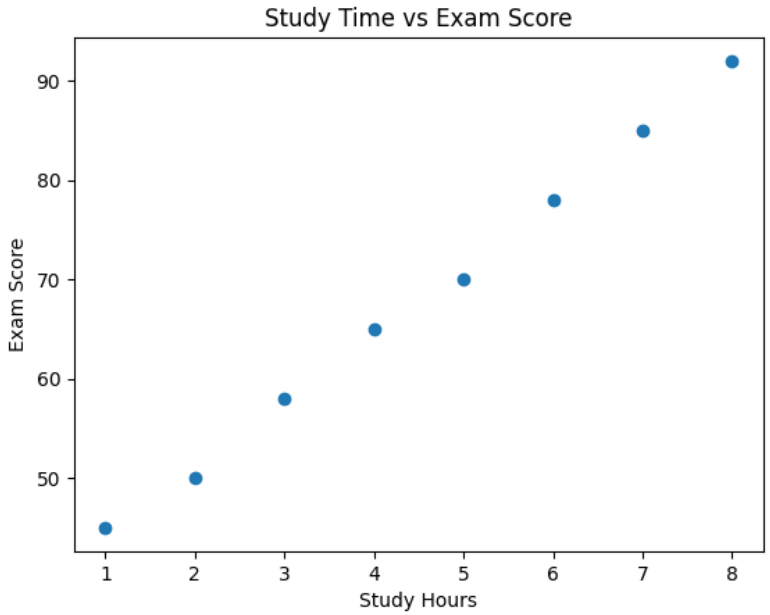
Aim
To identify the correlation between students' study time and exam scores.

Algorithm
→ Create study-time and score data.
→ Calculate the correlation coefficient.
→ Create a scatter plot.
→ Display the relationship.

Code
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Study Hours": [1,2,3,4,5,6,7,8],
    "Score": [45,50,58,65,70,78,85,92]
})

print("Correlation:", round(df["Study Hours"].corr(df["Score"]), 2))

plt.scatter(df["Study Hours"], df["Score"])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Time vs Exam Score")
plt.show()
```

Input
Students' study hours and exam scores.

Output
Correlation: 1.0



Result
The correlation between study time and exam scores was identified successfully.

38. Temperature Variability by City

Aim
To analyze temperature variability and consistency across cities.

Algorithm
→ Create temperature data for different cities.
→ Calculate mean and standard deviation.
→ Calculate temperature ranges.
→ Identify the city with highest range and lowest standard deviation.

Code
```
import pandas as pd

df = pd.DataFrame({
    "City": ["Chennai","Chennai","Chennai","Delhi","Delhi","Delhi",
             "Mumbai","Mumbai","Mumbai"],
    "Temperature": [30,32,31,20,35,25,28,30,29]
})

mean = df.groupby("City")["Temperature"].mean()
std = df.groupby("City")["Temperature"].std()
ranges = df.groupby("City")["Temperature"].agg(lambda x: x.max()-x.min())

print("Mean temperature:\n", mean)
print("\nStandard deviation:\n", std)
print("\nHighest temperature range:", ranges.idxmax())
print("Most consistent city:", std.idxmin())
```

Input
Daily temperature readings for each city.

Output
```
Mean temperature:
 City
Chennai    31.000000
Delhi      26.666667
Mumbai     29.000000
Name: Temperature, dtype: float64

Standard deviation:
 City
Chennai    1.000000
Delhi      7.637626
Mumbai     1.000000
Name: Temperature, dtype: float64

Highest temperature range: Delhi
Most consistent city: Chennai
```

Result
Temperature variability and consistency across cities were analyzed successfully.

---

39. K-Means E-Commerce Customer Clustering

Aim

To segment customers based on spending and purchase behavior and visualize the clusters.

Algorithm

→ Create customer spending and purchase data.
→ Apply K-Means clustering.
→ Assign customers to clusters.
→ Plot the clusters.
→ Display the results.

Code

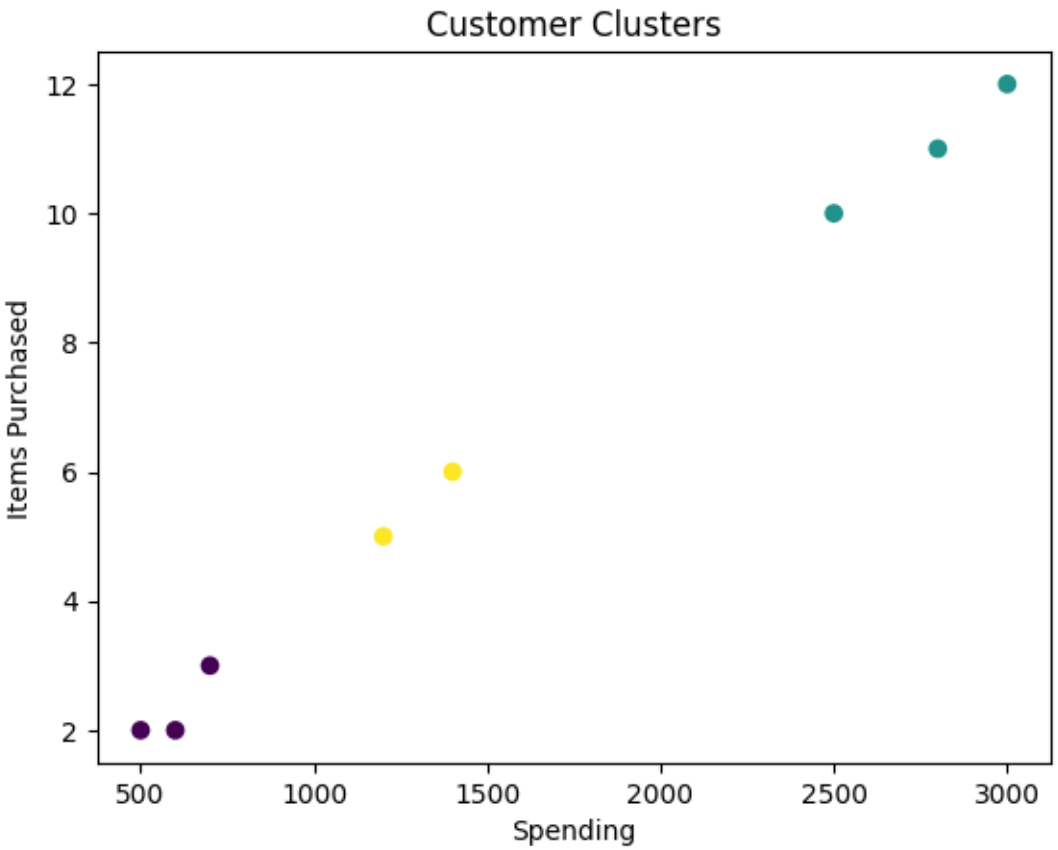
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

df = pd.DataFrame({
    "Customer": range(1,9),
    "Spending": [500,700,600,2500,3000,2800,1200,1400],
    "Items": [2,3,2,10,12,11,5,6]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)
df["Cluster"] = model.fit_predict(df[["Spending", "Items"]])

print(df)

plt.scatter(df["Spending"], df["Items"], c=df["Cluster"])
plt.xlabel("Spending")
plt.ylabel("Items Purchased")
plt.title("Customer Clusters")
plt.show()
```

Input

Customer spending and number of items purchased.

Output

```
   Customer  Spending  Items  Cluster
0         1       500      2        0
1         2       700      3        0
2         3       600      2        0
3         4      2500     10        1
4         5      3000     12        1
5         6      2800     11        1
6         7      1200      5        2
7         8      1400      6        2
```



Result
Customers were successfully segmented and the clusters were visualized.

---

40. Soccer Player Data Analysis

Aim
To analyze soccer player data and visualize player distribution by position.

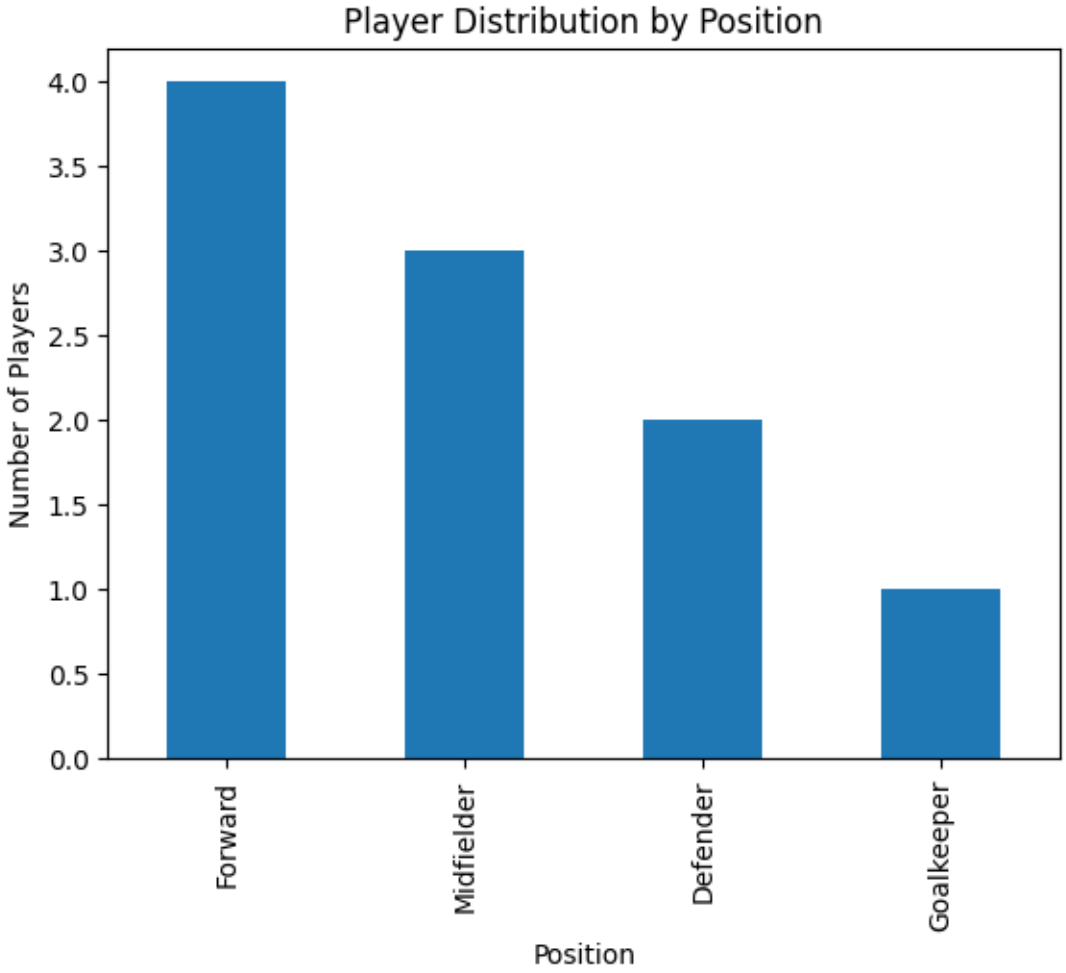
Algorithm
→ Create and store player data in a CSV file.
→ Read the CSV using Pandas.
→ Find the top five players by goals and salary.
→ Calculate average age and identify players above it.
→ Plot the distribution by position.

Code
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Name": ["A","B","C","D","E","F","G","H","I","J"],
    "Age": [22,25,28,30,24,27,31,26,29,23],
    "Position": ["Forward","Midfielder","Forward","Defender","Goalkeeper",
                 "Forward","Midfielder","Defender","Forward","Midfielder"],
    "Goals": [20,12,18,5,2,15,10,4,22,8],
    "Salary": [5000,4000,6000,3500,3000,5500,4500,3200,6500,3800]
})

df.to_csv("players.csv", index=False)
df = pd.read_csv("players.csv")

print("Top 5 by goals:")
print(df.nlargest(5, "Goals")[["Name", "Goals"]])

print("\nTop 5 by salary:")
print(df.nlargest(5, "Salary")[["Name", "Salary"]])

avg_age = df["Age"].mean()
print("\nAverage age:", round(avg_age, 2))
print("\nPlayers above average age:")
print(df[df["Age"] > avg_age]["Name"])

df["Position"].value_counts().plot(kind="bar")
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.title("Player Distribution by Position")
plt.show()
```

Input
Player name, age, position, goals scored, and weekly salary.

Output
```
Top 5 by goals:
  Name  Goals
8    I     22
0    A     20
2    C     18
5    F     15
```

```
1    B     12

Top 5 by salary:
  Name  Salary
8    I    6500
2    C    6000
5    F    5500
0    A    5000
6    G    4500

Average age: 26.5

Players above average age:
2    C
3    D
5    F
6    G
8    I
Name: Name, dtype: object
```



Result

The soccer player dataset was created, analyzed, and visualized successfully.